Multiple Critics and Learning Loops in Deep RL

Technion NeuroAI Lab

March 12, 2026

1 Introduction

Classical view. Reward-based learning in the brain is widely attributed to the midbrain dopamine system. Seminal work showed that dopamine neuron activity can resemble a *reward prediction error* (RPE): the difference between the reward received and the reward expected. This same signal appears in temporal-difference (TD) reinforcement learning algorithms. The classical view is that dopamine neurons transmit a *single*, *homogeneous* RPE that is *broadcast* to a large set of targets (e.g. corticostriatal synapses), so that one scalar error drives plasticity everywhere and implements a simple TD learner.

Conflicting evidence. Recent experiments have challenged this picture. In several conditions, dopamine activity is not well explained by a single broadcast RPE; in particular, during *complex* tasks the activity of dopamine neurons is *heterogeneous* and carries multiple signals related to different features of the environment. The same system can appear to transmit a homogeneous RPE in simple conditioning but highly heterogeneous signals in more complex, navigation-based decision-making. These findings suggest that the algorithm implemented by the brain is more complex than a single broadcast TD signal.

The dynamic learning loops hypothesis. To reconcile the data, we adopt the "dynamic learning loops" hypothesis (DopamineLearnLoops ERC proposal). The central claim is that the dopamine system does *not* broadcast one global learning signal that updates all targets equally. Instead, it is composed of *multiple learning modules* working in *parallel*, which transmit *different* learning signals to *different* brain areas. Each module can carry a learning signal that reflects a distinct aspect of the task (e.g. different state features or outcome dimensions). The signals can have both *shared* and *distinct* components. Crucially, the system is *dynamic*: as the learned behaviour becomes more complex (more relevant features, more actions), these modules become more engaged and transmit increasingly diverse signals. Parallelizing learning in this way can be advantageous when multiple features of the environment change independently or when learning complex functions; it may also support the kind of generalization that currently distinguishes biological from artificial learning.

Scope of this report. We implement a progression of deep RL algorithms on CartPole-v1 that build toward this idea. The following sections spell out the biological demands and implementation perspective, then the full mathematical details and the role of the feedback matrix B in each model.

1.1 Biological implementation and demands

What biology suggests. Several lines of evidence support the learning-loops view. (1) *Heterogeneous dopamine activity:* VTA dopamine neurons can show different functional responses, with subpopulations possibly serving different behavioural roles. (2) *Anatomical loops:* There are distinct functional loops linking subregions of the VTA to subregions of the striatum (e.g. lateral VTA with lateral NAcc, medial VTA with medial NAcc); such structure is consistent with partially segregated learning pathways rather than a single broadcast. (3) *Heterogeneous release:* Dopamine release in the striatum can vary by location and scale, implying that different targets receive different learning signals. (4) *Task-dependent dynamics:* In simple tasks, dopamine signals look homogeneous; in complex tasks they look heterogeneous—suggesting that the system adjusts how many distinct signals are active depending on task demands.

Biological demands on an algorithm. For an algorithm to be a plausible model of this system it should: (i) use *multiple* learning signals (not one scalar RPE) that can be routed to different "features" or subpopulations; (ii) allow these signals to reflect *different aspects* of the world (e.g. different state dimensions or outcome types); (iii) combine shared and distinct components across channels; (iv) ideally allow the *number* or *structure* of these channels to depend on task complexity (dynamic engagement). In addition, the routing from a high-level error (or errors) to the plasticity of earlier layers need not be implemented by full backpropagation through a deep stack; biology may use simpler, more local feedback. A fixed or structured *routing matrix* from error dimensions to feature dimensions is one way to implement multiple parallel loops without a single broadcast.

From biology to our models. In our computational models we interpret "multiple learning modules" as *multiple critics* (value functions or error channels) that evaluate different aspects of the task. The "routing" of their errors to a feature layer is implemented by a *feedback matrix* B: it maps an error vector $\mathbf{e}$ (one component per channel) to a learning signal $\boldsymbol{\delta}_z$ for the feature layer, so that each feature receives a prescribed linear combination of the channel errors. Thus B plays the role of the anatomical and functional structure that sends different learning signals to different targets. We do not yet implement dynamic engagement (variable number of critics with task complexity) or unsupervised discovery of aspects; we do implement multiple fixed channels and a fixed B, building from a single-critic baseline (DQN) to multi-head critics with trainable combination (multihead).

2 The CartPole-v1 task

Task description. CartPole-v1 is a classic control benchmark: a cart moves along a frictionless track and a pole is hinged on top of it. The agent must apply a horizontal force (left or right) at each time step to keep the pole upright and the cart within bounds. We use the Gymnasium implementation (`CartPole-v1`), which provides a standardised interface and reward structure.

State space. The observation is a vector of four continuous scalars (state s or $\mathbf{x}$ in our notation):

- **Cart position** $x \in [-4.8, 4.8]$ (units such that the track edges are at ± 2.4 for termination).
- **Cart velocity** $\dot{x}$ (unbounded).
- **Pole angle** θ (radians), approximately in $[-\pi/2 + 0.07, \pi/2 - 0.07]$; the pole is upright at $\theta = 0$.

- **Pole angular velocity** $\dot{\theta}$ (unbounded).

So $\mathbf{x} = (x,\, \dot{x},\, \theta,\, \dot{\theta})^\top \in \mathbb{R}^4$. The observation space is `Box` with shape $(4,)$, `float32`. In our multihead model we refer to "cart" as the pair $(x,\, \dot{x})$ and "pole" as $(\theta,\, \dot{\theta})$ when using masked state views.

Action space. `Discrete(2)`: action 0 = push cart left, action 1 = push cart right. The agent chooses one action per step.

Reward and episode length. The environment returns a reward of +1 for every step, including the step on which the episode ends. So the *episodic return* (sum of rewards in an episode) equals the number of steps in that episode. Episodes are *terminated* if the pole angle leaves the range $\pm 12°$ (approx. ± 0.2095 rad) or the cart position leaves $[-2.4,\, 2.4]$. Episodes are *truncated* if they reach 500 steps without terminating. Thus the maximum possible episodic return is 500 (an episode that lasts 500 steps and then is truncated). In our runs we use `RecordEpisodeStatistics` so that the return and length of each finished episode are available in the step info.

Implementation in our code. We create the environment with `gymnasium.make("CartPole-v1")` and wrap it with `RecordEpisodeStatistics`. Each training run is step-based: at each global step we (i) select an action (e.g. ϵ-greedy from the current Q-network or Q_{combined}), (ii) call `env.step(action)`, (iii) store the transition (s, a, r, s', d) in the replay buffer (with `done` = terminated or truncated, and we use the true next state for truncation when provided in `infos`), (iv) every `train_frequency` steps after `learning_starts`, sample a minibatch and perform a training update. When an episode ends we call `env.reset()` and continue. Observations are flattened to a vector of dimension `obs_dim` = 4 and fed to the network; no extra preprocessing is applied.

Requirements. The agent must learn a policy that, from arbitrary initial states (within the allowed ranges), keeps the pole upright and the cart inside the track for as many steps as possible. This requires (i) estimating which actions are good in which states (value function or Q-function), (ii) exploring sufficiently to visit informative state–action pairs, and (iii) using a replay buffer and target network (or similar) for stable off-policy learning. The environment is deterministic given the action; the main difficulty is credit assignment and exploration.

When the task is learned. We say the task is *solved* or *learned* when the agent consistently achieves the maximum episodic return of 500. The standard CartPole-v1 criterion is: **solved if the average return over 100 consecutive episodes is** ≥ 500. In practice we report **final return** (return of the last completed episode) and **mean return over the last 100 episodes** for each run; a run that has "learned" will show final return 500 and mean last 100 close to 500. Because the environment truncates at 500 steps, any run that reaches 500 steps per episode attains the maximum return; learning is successful when the policy reliably does so across episodes and seeds.

3 Background: Deep RL and Q-learning

Setting. An agent interacts with an environment over discrete steps. At step t it observes state s_t, chooses action a_t, receives reward r_t, next state s_{t+1}, and a done flag d_t (1 if the episode terminates or is truncated). The goal is to learn a policy that maximizes the cumulative discounted return $\sum_{k \geq 0} \gamma^k r_{t+k}$. We consider the standard setup where the Q-function is approximated by a neural network and trained off-policy from stored transitions.

Action-value and Bellman equation. The *action-value* $Q(s, a)$ is the expected return when taking action a in state s and then following the current policy. Under the optimal policy, Q^* satisfies the Bellman equation:

$$Q^*(s, a) = \mathbb{E}\big[r + \gamma\,(1 - d)\,\max_{a'} Q^*(s', a') \,\big|\, s, a\big]. \tag{1}$$

We approximate Q with a neural network and drive it toward this relation by minimizing the *temporal difference (TD) error*: the mismatch between the right-hand side (the *TD target*) and the current Q-estimate.

Two networks: online and target. In basic deep Q-learning we use **two** networks with the same architecture but different parameters:

- **Online network** (parameters θ): This is the network we train. It is used to (i) compute the current Q-values $Q(s, a; \theta)$ for each state–action pair, and (ii) select actions during interaction (e.g. $\arg\max_a Q(s, a; \theta)$ when not exploring). Only θ is updated by gradient descent.
- **Target network** (parameters θ^-): This network is *not* trained by gradient descent. It is a periodic copy of the online network ($\theta^- \leftarrow \theta$ every C steps, for some update interval C). It is used *only* to compute the Q-values of the *next* state s' when forming the TD target. Using a separate, slowly updated target avoids a moving target: the regression objective for the online network is fixed over many steps, which stabilizes learning.

So at any time there are two copies of the same architecture: one that is continuously updated (online) and one that is only updated by copying the online parameters every C steps (target).

TD target and loss. For a transition (s, a, r, s', d) sampled from the replay buffer, the **TD target** is computed using the *target* network:

$$y = r + \gamma\,(1 - d)\,\max_{a'} Q(s', a'; \theta^-). \tag{2}$$

The **current Q-estimate** for the taken action is computed using the *online* network: $Q(s, a; \theta)$. The **TD error** for that transition is $y - Q(s, a; \theta)$. We minimize the mean squared TD error over a minibatch $\mathcal{B}$ of transitions:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s',d)\in\mathcal{B}}\big[\big(y - Q(s, a; \theta)\big)^2\big]. \tag{3}$$

Only the online parameters θ appear in $\mathcal{L}$; y depends on θ^-, and we do *not* backpropagate through the target network. In practice we take one or more gradient steps on $\mathcal{L}$ with respect to θ (e.g. via Adam); θ^- is updated separately by the periodic copy $\theta^- \leftarrow \theta$. This is *deep Q-learning (DQN)*: a single scalar TD error per transition, and that error is broadcast through the online network via backpropagation to update all of θ.

Replay buffer and training loop. Transitions (s, a, r, s', d) are stored in a fixed-size *replay buffer*. When training, we sample a random minibatch $\mathcal{B}$ from the buffer and perform a gradient step on $\mathcal{L}(\theta)$ as above. Random sampling breaks temporal correlation and reuses each transition multiple times, improving sample efficiency.

Exploration. Actions are chosen ϵ-greedy with respect to the *online* network: with probability ϵ take a random action, otherwise take $\arg\max_a Q(s, a; \theta)$. Typically ϵ is annealed from 1 (or a high value) down to a small value (e.g. 0.05) over training, so that the agent explores heavily early and exploits the learned Q-values later.

4 Backpropagation and weight updates

In deep Q-learning the loss $\mathcal{L}(\theta)$ is minimized by gradient descent. The gradients are computed by **backpropagation**: a forward pass computes the loss and stores intermediate activations; a backward pass applies the chain rule layer by layer to obtain $\partial\mathcal{L}/\partial\theta$. We summarize how the forward pass, backward pass, and weight updates work in a generic feedforward network, then relate this to DQN and to our mixed variants.

Forward pass. Consider a network that maps input $\mathbf{x}$ to output $\mathbf{o}$ (e.g. Q-values) through L layers. Layer ℓ has parameters $W^{(\ell)}$ (and optionally bias $b^{(\ell)}$), input activation $\mathbf{a}^{(\ell-1)}$ (with $\mathbf{a}^{(0)} = \mathbf{x}$), and output activation $\mathbf{a}^{(\ell)} = f^{(\ell)}(W^{(\ell)}\mathbf{a}^{(\ell-1)} + b^{(\ell)})$ for some nonlinearity $f^{(\ell)}$ (e.g. ReLU or identity). The final layer output $\mathbf{a}^{(L)}$ is used to form the loss $\mathcal{L}$ (e.g. $\mathcal{L} = (y - Q(s, a))^2$ where $Q(s, a)$ is one component of $\mathbf{a}^{(L)}$). During the forward pass we *store* all intermediate activations $\mathbf{a}^{(\ell)}$ and pre-activations $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + b^{(\ell)}$; these are needed in the backward pass to compute gradients.

Backward pass. The backward pass computes the gradient of $\mathcal{L}$ with respect to every parameter. We define the **error signal** (or "delta") at layer ℓ as $\boldsymbol{\delta}^{(\ell)} = \frac{\partial\mathcal{L}}{\partial\mathbf{z}^{(\ell)}} \in \mathbb{R}^{\dim(\ell)}$. At the output layer, $\boldsymbol{\delta}^{(L)}$ is obtained directly from $\partial\mathcal{L}/\partial\mathbf{a}^{(L)}$ and the derivative of $f^{(L)}$. Then we propagate backwards: $\boldsymbol{\delta}^{(\ell-1)} = (W^{(\ell)})^\top\boldsymbol{\delta}^{(\ell)} \odot f'^{(\ell-1)}(\mathbf{z}^{(\ell-1)})$, where $\odot$ is element-wise product and f' is the derivative of the nonlinearity (e.g. for ReLU, $f'(z) = 1$ if $z > 0$ and 0 otherwise). This is the **chain rule**: the error at the output flows backward through the weights and nonlinearities. Once we have $\boldsymbol{\delta}^{(\ell)}$, the gradients for the parameters of layer ℓ are:

$$\frac{\partial\mathcal{L}}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)\top}\,\mathbf{a}^{(\ell-1)}, \qquad \frac{\partial\mathcal{L}}{\partial b^{(\ell)}} = \boldsymbol{\delta}^{(\ell)\top}. \tag{4}$$

(For a minibatch, these become matrices: $\partial\mathcal{L}/\partial W^{(\ell)} = \sum_{b\in\mathcal{B}} \boldsymbol{\delta}_b^{(\ell)}\mathbf{a}_b^{(\ell-1)\top}$ and similarly for b, then averaged or summed over the batch.) So each weight $W_{ij}^{(\ell)}$ is updated using the error signal at unit j and the activation of the incoming unit i: the same scalar TD error is *broadcast* through all layers, and every parameter receives a gradient that depends on the full path from that parameter to the loss.

Weight update. After the backward pass we have $\nabla_\theta\mathcal{L} = (\ldots, \partial\mathcal{L}/\partial W^{(\ell)}, \partial\mathcal{L}/\partial b^{(\ell)}, \ldots)$. Parameters are updated by gradient descent: $\theta \leftarrow \theta - \eta\,\nabla_\theta\mathcal{L}$ (or by Adam: a running estimate of first and second moments of the gradient is maintained, and the update is a scaled step in the direction of the gradient). So the **online** network's weights change in the direction that reduces the TD error on the current minibatch; the **target** network is not updated by gradients but by copying θ periodically.

Summary. In standard DQN, a single scalar TD error $y - Q(s, a; \theta)$ is the only loss. The forward pass computes $Q(s, a; \theta)$ from $\mathbf{x}$ through all layers and stores activations. The backward pass

propagates $\partial\mathcal{L}/\partial Q$ (proportional to the TD error) backward through every layer, producing $\boldsymbol{\delta}^{(\ell)}$ at each layer and hence $\partial\mathcal{L}/\partial W^{(\ell)}$ and $\partial\mathcal{L}/\partial b^{(\ell)}$. Every weight is updated using this full backward flow. In the next section we describe how our mixed variants *break* this flow at the feature layer and replace it with a prescribed learning signal $\boldsymbol{\delta}_z = B\mathbf{e}$.

5 The feedback matrix B and its role

In standard DQN there is no matrix B: a single scalar TD error is used in the loss, and its gradient flows through the *entire* network via backpropagation. In our *mixed backprop* variants we split the network into a *linear feature layer* (output $\mathbf{z} \in \mathbb{R}^F$) and a *nonlinear trunk* that maps $\mathbf{z}$ to Q-values. The trunk is trained by normal backprop on the TD loss. The feature layer is *not* updated by gradients through the trunk; instead it receives a *learning signal* $\boldsymbol{\delta}_z \in \mathbb{R}^F$ built from an *error vector* $\mathbf{e}$ at the Q-output. The **feedback matrix** B is a fixed (non-learned) matrix that defines how $\mathbf{e}$ is turned into $\boldsymbol{\delta}_z$: $\boldsymbol{\delta}_z = B\mathbf{e}$. Below we give the full mathematical details of forward pass, backward pass, and training for each part of the network, then discuss biological plausibility.

Architecture common to mixed variants. Let the state (observation) be $\mathbf{x} \in \mathbb{R}^n$ (e.g. $n = 4$ for CartPole). The network has:

- **Linear feature layer:** $\mathbf{z} = W_z\mathbf{x} + b_z \in \mathbb{R}^F$, with $W_z \in \mathbb{R}^{F\times n}$, $b_z \in \mathbb{R}^F$.
- **Trunk:** A feedforward network (e.g. ReLU hidden layers) that maps $\mathbf{z}$ to an internal representation $\mathbf{h}$, then a readout to $\mathbf{Q} \in \mathbb{R}^A$ (or to multiple heads). So $Q(s, a)$ is the a-th component of the final output.

During *training* of the mixed variants we use $\mathbf{z}$ with `detach`: the trunk receives $\mathbf{z}_{\text{detach}}$ so that no gradient flows from the loss back through W_z via the trunk. The feature layer is updated by a *separate* rule using B and $\mathbf{e}$.

Forward pass. For a minibatch of states $\mathbf{x}_b$:

1. **Feature layer:** $\mathbf{z}_b = W_z\mathbf{x}_b + b_z$. We store $\mathbf{z}_b$ (and $\mathbf{x}_b$) for the updates.
2. **Trunk input:** The trunk receives either $\mathbf{z}_b$ (standard DQN) or $\mathbf{z}_b$ with gradients detached (mixed). It computes $\mathbf{h}_b = \text{trunk}(\mathbf{z}_b)$ and then $\mathbf{Q}_b = \text{readout}(\mathbf{h}_b)$ (or multiple head outputs). So $Q(s, a)$ is available for the taken action a.
3. **TD target:** $y_b = r_b + \gamma(1 - d_b)\max_{a'} Q^-(s'_b, a')$ using the *target* network (same forward pass with θ^-).
4. **Loss:** $\mathcal{L} = \frac{1}{|\mathcal{B}|}\sum_b (y_b - Q(s_b, a_b; \theta))^2$. Only the online network's Q-output and trunk parameters participate in this loss; W_z does *not* receive gradients from $\mathcal{L}$ in mixed variants because the trunk input was detached.

Error vector e. From the TD targets and current Q-values we form an **error vector e** per sample (or per minibatch). Its dimension and definition depend on the model:

- **DQN:** No $\mathbf{e}$; a single scalar $y - Q(s, a)$ is the only error; all layers get gradients from the same scalar.

- **Mixed:** $\mathbf{e} \in \mathbb{R}^A$: $e_a = y - Q(s,a)$ for the taken action a, and $e_{a'} = 0$ for $a' \neq a$.
- **Mixed 3F:** $\mathbf{e} \in \mathbb{R}^K$, $K = 3F$: for the taken action a, the TD error $y - Q(s,a)$ is spread over $G = K/A$ indices $\mathcal{G}_a$ so that $e_j = (y - Q(s,a))/G$ for $j \in \mathcal{G}_a$ and $e_j = 0$ otherwise (so $\sum_{j \in \mathcal{G}_a} e_j = y - Q(s,a)$).
- **Multihead:** $\mathbf{e} \in \mathbb{R}^5$: $e_k = y_k - Q_k(s,a)$ for each head k, with head-specific targets y_k.

Learning signal for the feature layer. The **feedback matrix** B (fixed, not learned) maps the error vector to a learning signal for the F features:

$$\boldsymbol{\delta}_z = B\mathbf{e}, \qquad \text{so} \quad \delta_{z,i} = \sum_j B_{ij} e_j. \tag{5}$$

So $\delta_{z,i}$ is the learning signal for feature i; it is a linear combination of the components of $\mathbf{e}$. The structure of B per model:

- **Mixed:** $B \in \mathbb{R}^{F \times A}$, sparse: $(B)_{i,\, i \bmod A} = 1$, rest 0. So $\delta_{z,i} = e_{i \bmod A}$ (each feature gets one action's error).
- **Mixed 3F:** $B \in \mathbb{R}^{F \times K}$: $(B)_{i,3i} = (B)_{i,3i+1} = (B)_{i,3i+2} = 1/3$, rest 0. So $\delta_{z,i} = \frac{1}{3}(e_{3i} + e_{3i+1} + e_{3i+2})$.
- **Multihead:** $B \in \mathbb{R}^{F \times 5}$: $(B)_{i,\, i \bmod 5} = 1$, rest 0. So each feature gets one of the five head errors.

Backward pass and updates—trunk. The trunk (and readout, and in multihead the heads and combination weights) is updated by **standard backpropagation** on the TD loss $\mathcal{L}$ (and in multihead, the sum of per-head losses plus combined loss). So:

$$\frac{\partial \mathcal{L}}{\partial(\text{trunk params})} \quad \text{computed by backprop from} \quad \frac{\partial \mathcal{L}}{\partial Q}; \qquad \text{trunk params} \leftarrow \text{trunk params} - \eta \nabla \mathcal{L}. \tag{6}$$

The error signal at the Q-output (the TD error) is propagated backward through the trunk only; it does *not* reach W_z because $\mathbf{z}$ was detached.

Backward pass and updates—feature layer. The feature layer is *not* updated by backprop through the trunk. Instead we use $\boldsymbol{\delta}_z = B\mathbf{e}$ as the *prescribed* error signal at the feature layer. Treating $\boldsymbol{\delta}_z$ as the gradient of some loss with respect to $\mathbf{z}$, the gradient of that (virtual) loss with respect to W_z and b_z would be, by the same logic as in the backprop section (with $\mathbf{z} = W_z\mathbf{x} + b_z$):

$$\nabla_{W_z} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \boldsymbol{\delta}_{z,b}^\top \mathbf{x}_b = \frac{1}{|\mathcal{B}|} \boldsymbol{\delta}_z^\top \mathbf{X}, \qquad \nabla_{b_z} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \boldsymbol{\delta}_{z,b} = \overline{\boldsymbol{\delta}_z}. \tag{7}$$

Here $\mathbf{X}$ is the minibatch of observations and $\boldsymbol{\delta}_{z,b} = (B\mathbf{e}_b)$; overloading notation, $\boldsymbol{\delta}_z$ is the matrix of learning signals (one row per sample). The update is:

$$W_z \leftarrow W_z + \eta_z \nabla_{W_z}, \qquad b_z \leftarrow b_z + \eta_z \nabla_{b_z}, \tag{8}$$

often with gradient-norm clipping (e.g. scale ∇_{W_z} so its norm does not exceed a constant). So the feature layer is updated using *only* the learning signal $\boldsymbol{\delta}_z = B\mathbf{e}$ and the input $\mathbf{x}$; no gradient flows through the trunk. This implements **multiple parallel learning loops**: each feature i receives $\delta_{z,i}$, which is a fixed linear combination of the error channels e_j, and the weight row $W_{z,i}$ is updated proportionally to $\delta_{z,i}\mathbf{x}$.

Summary table. In DQN, one scalar TD error drives the entire network via one backward pass. In mixed variants, the trunk gets one backward pass from the TD loss; the feature layer gets a *separate* update using $\boldsymbol{\delta}_z = B\mathbf{e}$ and $\mathbf{x}$, with no gradient through the trunk. The matrix B fixes how error dimensions (actions or heads) are routed to feature dimensions.

Biological plausibility—why it is plausible. (1) **Multiple learning signals:** The brain is thought to use multiple, partially distinct learning signals (e.g. heterogeneous dopamine) rather than one broadcast RPE; our $\mathbf{e}$ has multiple components and B routes them to different features. (2) **Fixed routing:** The matrix B is a fixed wiring from "error channels" to "feature units," analogous to anatomical connectivity from different dopamine subpopulations to different striatal regions. (3) **No full backprop through a deep stack:** The feature layer is updated by a simple, local rule ($\boldsymbol{\delta}_z^\top \mathbf{x}$) that does not require propagating gradients through the trunk; biology may use similar local, target-derived teaching signals rather than exact backprop through many layers. (4) **Composition of errors:** Each feature receives a linear combination of errors ($\delta_{z,i} = \sum_j B_{ij} e_j$), consistent with convergent dopaminergic projections and combined plasticity signals.

Biological plausibility—why it is not. (1) **Trunk still uses backprop:** The trunk (and readout) are trained by full backpropagation; if the trunk is deep, this still requires the kind of global, layer-by-layer gradient flow that is often argued to be biologically implausible. (2) **Precise error signals:** The TD errors e_k are exact scalar differences (target minus current Q); real neurons may carry noisier or coarser signals. (3) **Synchronous updates and perfect storage:** We assume minibatches, exact replay, and synchronous parameter updates; biology has asynchronous plasticity and no literal replay buffer. (4) **Fixed B:** We do not learn B; in the brain, the strength of projections from dopamine to striatum may be plastic. (5) **Detachment is a design choice:** Using $\mathbf{z}_{\text{detach}}$ is a way to implement two learning pathways in software; it does not by itself imply a specific neural mechanism. So the mixed setup is a *step* toward biological plausibility (multiple loops, local feature-layer update) but still relies on backprop in the trunk and on idealized error and update rules.

6 Model 1: CleanRL DQN

Philosophy. This is the baseline: a *single critic* and a *single learning loop*. One scalar TD error is computed and broadcast through the entire network via backpropagation. There is no split between "feature" and "trunk," no feedback matrix B, and no multiple error channels. It represents the classical view of dopamine as one homogeneous RPE driving plasticity everywhere.

Network components. The network is a single feedforward MLP with parameters θ (one copy for the online network, one for the target θ^-). There are no logically separate submodules:

- **Input:** State $\mathbf{x} \in \mathbb{R}^n$ (e.g. $n = 4$ for CartPole).
- **Hidden layer 1:** Linear map $\mathbb{R}^n \to \mathbb{R}^{120}$ then ReLU. No separate name for the output; it is just the first hidden activation.
- **Hidden layer 2:** Linear map $\mathbb{R}^{120} \to \mathbb{R}^{84}$ then ReLU.
- **Output (readout):** Linear map $\mathbb{R}^{84} \to \mathbb{R}^A$. The a-th component is $Q(s, a; \theta)$.

Forward pass: $\mathbf{x} \to \text{ReLU}(W_1\mathbf{x} + b_1) \to \text{ReLU}(W_2 \cdot) \to W_3 \cdot \to \mathbf{Q} \in \mathbb{R}^A$.

Error signal and training. There is a single TD error per transition: $y - Q(s, a; \theta)$, with $y = r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta^-)$. The loss is $\mathcal{L} = \mathbb{E}_{\mathcal{B}}[(y - Q(s, a; \theta))^2]$. **All** parameters θ (both hidden layers and readout) are updated by one backward pass: the gradient of $\mathcal{L}$ flows from the output back through every layer. The target network θ^- is updated by copying θ every 500 steps; it is used only in the forward pass to compute y, never in the backward pass. So one scalar error drives one global update.

7 Model 2: Mixed backprop (simple)

Philosophy. We introduce a *split* between a *feature layer* and a *trunk*, and a *second learning pathway* for the feature layer that does not use backprop through the trunk. The feature layer is updated by a prescribed learning signal $\boldsymbol{\delta}_z = B\mathbf{e}$, where $\mathbf{e}$ is an error vector (here, one non-zero component per action). So we have *multiple error dimensions* (A actions) routed to F features via a fixed matrix B, giving multiple parallel learning loops while the trunk still learns by standard backprop.

Network components. Same total dimensions as DQN (120, 84, A) but with a clear functional split:

- **Input:** State $\mathbf{x} \in \mathbb{R}^n$.
- **Feature layer (linear):** $\mathbf{z} = W_z\mathbf{x} + b_z \in \mathbb{R}^F$ with $F = 120$. This is the only layer that will be updated by $B\mathbf{e}$ instead of backprop.
- **Trunk:** A feedforward block that maps $\mathbf{z}$ to a hidden representation: $\mathbf{h} = \mathrm{ReLU}(W_h\mathbf{z} + b_h) \in \mathbb{R}^{84}$. During training of the mixed variant, the trunk can receive $\mathbf{z}$ with gradients detached so that no gradient from the loss flows back into W_z through the trunk.
- **Readout:** Linear map $\mathbb{R}^{84} \to \mathbb{R}^A$; $Q(s, a)$ is the a-th component.

Forward pass: $\mathbf{x} \to \mathbf{z} \to \mathbf{h} \to \mathbf{Q}$. For action selection we use the full forward pass (no detach).

Error vector and feedback matrix. For the taken action a, we set $e_a = y - Q(s, a)$ and $e_{a'} = 0$ for $a' \neq a$, so $\mathbf{e} \in \mathbb{R}^A$. The fixed matrix $B \in \mathbb{R}^{F \times A}$ is sparse: $(B)_{i, i \bmod A} = 1$, all other entries 0. Hence $\boldsymbol{\delta}_z = B\mathbf{e}$ gives $\delta_{z,i} = e_{i \bmod A}$: feature i receives the TD error of action i mod A.

Training. (1) **Trunk and readout:** Updated by backprop on $\mathcal{L} = (y - Q(s, a))^2$; the TD error flows backward through the readout and trunk only (and, in implementation, the trunk input is $\mathbf{z}_{\text{detach}}$ so gradients do not reach W_z). (2) **Feature layer:** $\nabla_{W_z} = \frac{1}{|\mathcal{B}|} \sum \boldsymbol{\delta}_z^\top \mathbf{x}$, $\nabla_{b_z} = \frac{1}{|\mathcal{B}|} \sum \boldsymbol{\delta}_z$; then $W_z \leftarrow W_z + \eta_z \nabla_{W_z}$, $b_z \leftarrow b_z + \eta_z \nabla_{b_z}$, with gradient-norm clipping. Target network is copied every 250 steps.

8 Model 3: Mixed 3F

Philosophy. We keep the same split (feature layer updated by $B\mathbf{e}$, trunk by backprop) but increase the *number of error channels* by making the trunk output a larger vector $\mathbf{q}_{\text{raw}} \in \mathbb{R}^K$ with $K = 3F$. The Q-values are obtained by averaging over groups of these channels. The error vector $\mathbf{e}$ has K components (one per channel for the taken action's group). The feedback matrix B is chosen

so that each *feature* receives a combination of *three* error channels (e.g. $\delta_{z,i} = \frac{1}{3}(e_{3i}+e_{3i+1}+e_{3i+2})$). So each feature is driven by a blend of several error dimensions, rather than a single action error as in Model 2.

Network components.

- **Input:** State $\mathbf{x} \in \mathbb{R}^n$.
- **Feature layer:** $\mathbf{z} = W_z\mathbf{x} + b_z \in \mathbb{R}^F$ ($F = 120$). Same role as in Model 2; updated only via $\boldsymbol{\delta}_z = B\mathbf{e}$.
- **Trunk:** Maps $\mathbf{z}$ to an internal representation (e.g. $\mathbf{h} \in \mathbb{R}^{84}$) then to a *raw* output $\mathbf{q}_{\text{raw}} \in \mathbb{R}^K$ with $K = 3F = 360$. There is no single readout to A actions; instead, the K dimensions are grouped by action.
- **Q-values by grouping:** For action a, $Q(s,a) = \frac{1}{G}\sum_{j\in\mathcal{G}_a}(\mathbf{q}_{\text{raw}})_j$ with $G = K/A$ and $\mathcal{G}_a = \{aG, \ldots, (a+1)G-1\}$. So each Q-value is the mean of G trunk outputs.

Forward pass: $\mathbf{x} \to \mathbf{z} \to \mathbf{h} \to \mathbf{q}_{\text{raw}} \to Q(s,a)$ (by averaging over $\mathcal{G}_a$).

Error vector and feedback matrix. For the taken action a, set $e_j = (y - Q(s,a))/G$ for $j \in \mathcal{G}_a$ and $e_j = 0$ otherwise; thus $\mathbf{e} \in \mathbb{R}^K$ and $\sum_{j\in\mathcal{G}_a} e_j = y - Q(s,a)$. The fixed matrix $B \in \mathbb{R}^{F\times K}$ has $(B)_{i,3i} = (B)_{i,3i+1} = (B)_{i,3i+2} = 1/3$, rest 0, so $\delta_{z,i} = \frac{1}{3}(e_{3i} + e_{3i+1} + e_{3i+2})$. Each feature gets a combination of three error channels.

Training. Trunk is trained by backprop on $\mathcal{L} = (y - Q(s,a))^2$ (gradients flow through the grouping operation). Feature layer is updated as in Model 2: ∇_{W_z}, ∇_{b_z} from $\boldsymbol{\delta}_z = B\mathbf{e}$, then W_z, b_z updated with η_z and clipping. Target network every 250 steps.

9 Model 4: Multihead (five critics)

Philosophy. This model implements **multiple critics** explicitly: five separate value heads, each with its own TD target and hence its own learning signal. The heads can differ in what they predict (reward, uprightness, short horizon) and in which part of the state they see (full state, cart-only, pole-only). A *trainable combination* w mixes the five head outputs for action selection. The feature layer is updated by $\boldsymbol{\delta}_z = B\mathbf{e}$ where $\mathbf{e} \in \mathbb{R}^5$ contains the five head TD errors; B routes each head's error to a subset of features. So we have five parallel learning loops (five critics) and a single shared feature representation that receives a structured combination of their errors via B.

Network components.

- **Input:** State $\mathbf{x} \in \mathbb{R}^n$ (e.g. $(x, \dot{x}, \theta, \dot{\theta})$ for CartPole).
- **Feature layer:** $\mathbf{z} = W_z\mathbf{x} + b_z \in \mathbb{R}^F$ ($F = 120$). Shared by all heads; updated only via $\boldsymbol{\delta}_z = B\mathbf{e}$.
- **Trunk:** Shared hidden representation $\mathbf{h} \in \mathbb{R}^{84}$ (ReLU layers). No single readout to $\mathbb{R}^A$; instead, multiple heads read from $\mathbf{h}$ (and possibly from masked state).

- **Five heads:** Each head $k \in \{1, \ldots, 5\}$ is a linear (or affine) readout producing $Q_k(s, a)$ for all actions a. Heads 1–3 use the full state (via $\mathbf{h}$); head 4 uses a *cart-masked* state $(x, \dot{x}, 0, 0)$; head 5 uses *pole-masked* state $(0, 0, \theta, \dot{\theta})$. So different heads can specialize to different aspects of the task.
- **Combination weights:** Trainable logits parametrize $w_k = \text{softmax}(\text{logits})_k$. The **combined Q** used for action selection is $Q_{\text{combined}}(s, a) = \sum_{k=1}^{5} w_k Q_k(s, a)$.

Forward pass: $\mathbf{x} \to \mathbf{z} \to \mathbf{h}$; then each head computes $Q_k(s, a)$ from $\mathbf{h}$ (and its state view); then $Q_{\text{combined}} = \sum_k w_k Q_k$.

Five TD targets. Each head has its own target, so five different "critics" learning different objectives:

$$y_1 = r + \gamma\,(1-d)\,\max_{a'} Q_1^-(s', a'), \qquad y_2 = \cos\theta' + \gamma\,(1-d)\,\max_{a'} Q_2^-(s', a'), \tag{9}$$

$$y_3 = r + \gamma_{\text{short}}\,(1-d)\,\max_{a'} Q_3^-(s', a'), \qquad y_4 = r + \gamma\,(1-d)\,\max_{a'} Q_4^-(s'_{\text{cart}}, a'), \tag{10}$$

$$y_5 = r + \gamma\,(1-d)\,\max_{a'} Q_5^-(s'_{\text{pole}}, a'). \tag{11}$$

Here $\gamma = 0.99$, $\gamma_{\text{short}} = 0.9$; $\cos\theta'$ is an uprightness outcome; s'_{cart} and s'_{pole} are the next state with pole or cart components zeroed.

Error vector and feedback matrix. For the taken action a, $e_k = y_k - Q_k(s, a)$ for $k = 1, \ldots, 5$, so $\mathbf{e} \in \mathbb{R}^5$. The fixed matrix $B \in \mathbb{R}^{F \times 5}$ has $(B)_{i,\, i \bmod 5} = 1$, rest 0. Thus $\boldsymbol{\delta}_z = B\mathbf{e}$: feature i receives the TD error of head $i \bmod 5$. So the five critics' errors are routed in a fixed pattern to the F features.

Training. (1) **Trunk and all five heads and combination logits:** One backward pass on the sum of the five per-head MSE losses $\mathcal{L}_k = (y_k - Q_k(s, a))^2$ plus the combined loss $\mathcal{L}_{\text{combined}} = (y_{\text{combined}} - Q_{\text{combined}}(s, a))^2$ with $y_{\text{combined}} = r + \gamma(1-d)\max_{a'} Q^-_{\text{combined}}(s', a')$. (2) **Feature layer:** Updated as in Models 2 and 3 using $\boldsymbol{\delta}_z = B\mathbf{e}$ and $\mathbf{x}$, with η_z and gradient-norm clipping. Target network every 250 steps.

10 Experimental Setup

Environment: CartPole-v1 (Gymnasium). We run each algorithm for 500 000 steps with seeds 1 and 2. Replay buffer size 10 000; batch size 128; training starts after 10 000 steps and every 10 steps thereafter. Discount $\gamma = 0.99$ (and $\gamma_{\text{short}} = 0.9$ for multihead head 3). ϵ-greedy: ϵ decreases linearly from 1 to 0.05 over the first 250 000 steps. Learning rate 2.5×10^{-4} (Adam) for the main network; for mixed variants the linear-feature layer uses $\eta_z = 10^{-4}$ and gradient norm clip 1. Target network updated every 500 steps (DQN) or 250 steps (mixed, mixed3f, multihead). Implementation: `run_cleanrl_vs_mixed_dqn.py`.

11 Results

11.1 Learning curves

Figure 1 shows episodic return versus episode for all four algorithms (individual seeds and mean). CleanRL DQN and Mixed 3F reach the maximum return (500) for both seeds; multihead and mixed

show more variance across seeds.

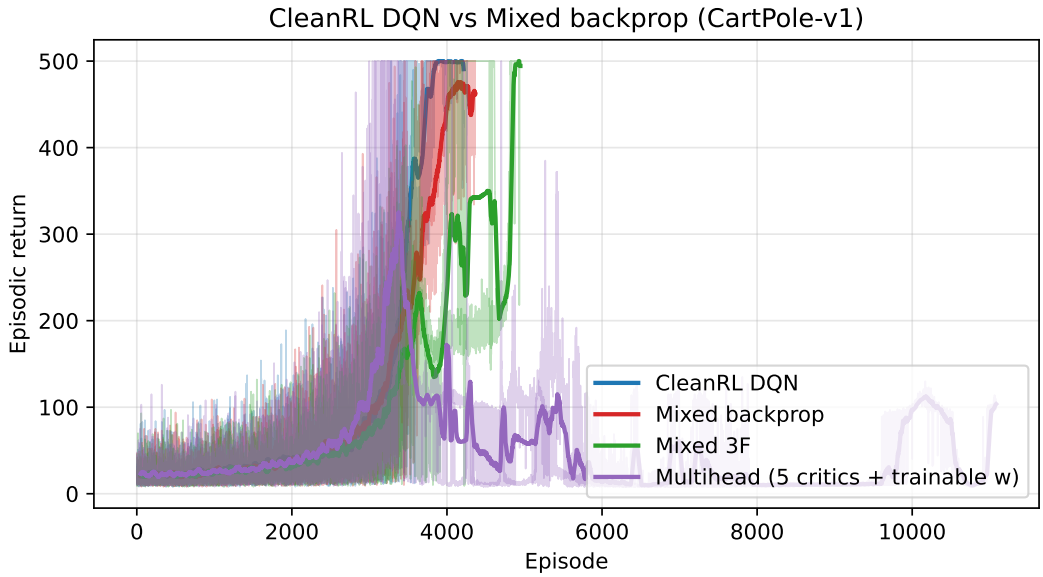

Figure 1: Episodic return over training. DQN, mixed backprop, mixed 3F, and multihead (5 critics + trainable w) on CartPole-v1 (500k steps, 2 seeds).

11.2 Training loss

Figure 2 shows the mean TD loss per episode for each algorithm.

11.3 Results table

Table 1 reports final return, mean return over the last 100 episodes, and training time per run. To regenerate: run the training script with `--algos dqn,mixed,mixed3f,multihead`, then `python code/make_report_table.py`, then recompile this document.

Table 1: Results: final return, mean return over last 100 episodes, and training time.

Algorithm	Seed	Final return	Mean last 100	Time (s)
CleanRL DQN	1	424.0	494.5	233
CleanRL DQN	2	500.0	500.0	226
Mixed backprop	1	500.0	450.3	234
Mixed backprop	2	500.0	499.9	240
Mixed 3F	1	500.0	495.3	462
Mixed 3F	2	321.0	452.7	484
Multihead	1	103.0	100.2	748
Multihead	2	35.0	29.9	738

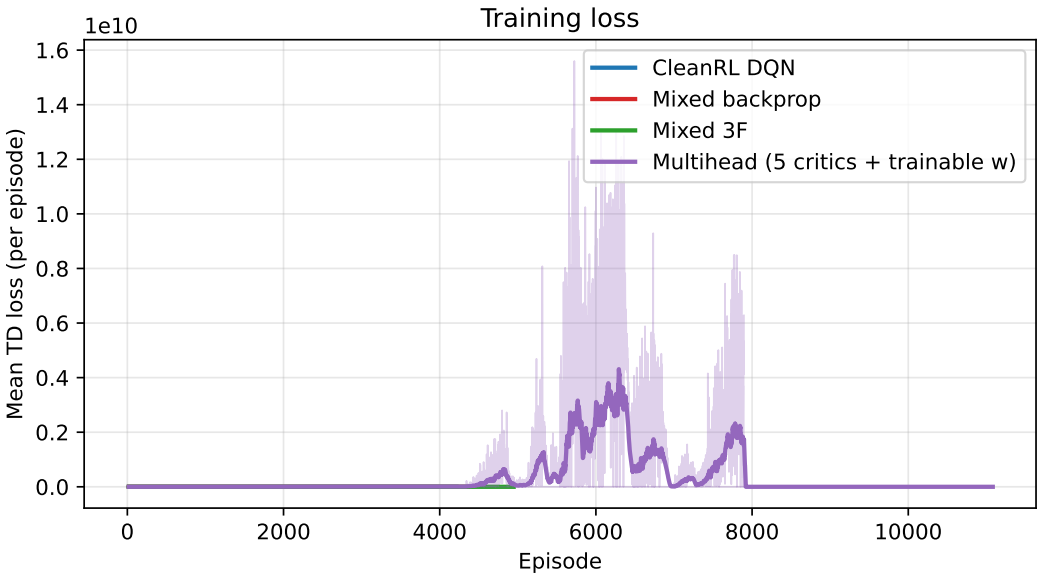

Figure 2: Mean TD loss per episode. Same four algorithms.

11.4 Biological interpretation of the models

The four models form a progression toward the dopamine learning-loops hypothesis. Below we spell out the biological implications of each, why it is structured that way, and what is gained relative to the previous model.

Model 1 (DQN): single broadcast. DQN corresponds to the *classical* view of dopamine: one scalar RPE $(y - Q(s, a))$ is computed and broadcast through the entire network via backpropagation. Every synapse is updated by the same global error. There is no anatomical or functional split between "error channels" and "feature units"—only one learning loop. Biologically, this would imply that all striatal (or cortical) targets receive the same dopamine signal and that a single homogeneous RPE suffices to explain plasticity. The structure is minimal by design: we need a baseline that matches the standard TD story and that performs well, so that any change we make in later models can be interpreted as adding structure rather than fixing a broken algorithm. **Gain over nothing:** a working, interpretable single-critic baseline that fits the classical DA picture.

Model 2 (Mixed): multiple error dimensions, fixed routing. We introduce a *split* between a feature layer (interpreted as an early representation, e.g. striatal or cortical input layer) and a trunk (deeper processing). Crucially, the feature layer is no longer updated by the global backpropagated error; instead it receives a *learning signal* $\boldsymbol{\delta}_z = B\mathbf{e}$ where $\mathbf{e}$ has one non-zero component per action. So we have **multiple error dimensions** (one per action) and a **fixed routing** B that sends each action's TD error to a prescribed subset of features. Biologically, this corresponds to the idea that different "dopamine channels" (e.g. different subpopulations or release sites) could carry errors that are still reward-based but *routed* to different striatal subregions or feature units. The same scalar TD error is no longer broadcast to everyone: feature i gets only $e_{i \bmod A}$, so different features are driven by different action errors. The trunk continues to learn by full backprop, so the "downstream" policy still benefits from a single coherent gradient; the gain is at the *feature* level, where we now have parallel, routed learning loops. **Gain over DQN:** (i) Multiple learning signals

(one per action dimension) instead of one; (ii) fixed routing B as a first step toward anatomical segregation; (iii) the feature layer is updated by a local rule ($\boldsymbol{\delta}_z^\top \mathbf{x}$) without backprop through the trunk, which is more plausible for biology than full deep backprop to the earliest layer.

Model 3 (Mixed 3F): blending error channels at each feature. We keep the same split (feature layer updated by $B\mathbf{e}$, trunk by backprop) but increase the number of error channels ($K = 3F$) and let each *feature* receive a *combination* of several error channels (e.g. $\delta_{z,i} = \frac{1}{3}(e_{3i} + e_{3i+1} + e_{3i+2})$). So each feature is no longer driven by a single action's error; it is driven by a blend of three channels. Biologically, this matches the idea that a given striatal unit might receive convergent input from *several* dopamine sources or that the same RPE is distributed over multiple channels that then converge onto the same plasticity site. The structure is chosen to test whether "combined" learning signals at each feature (shared and distinct components) can still support good performance. **Gain over Mixed:** (i) Each feature receives a linear combination of errors (shared and distinct components), closer to the biological demand that signals "combine shared and distinct components across channels"; (ii) we can study whether blending multiple error dimensions at the feature level helps or hurts, as a step toward more realistic convergent DA projections.

Model 4 (Multihead): multiple critics and different aspects. Here we implement **multiple critics** explicitly: five value heads, each with its own TD target and hence its own learning signal. The heads can differ in what they predict (reward return, uprightness, short horizon) and in which part of the state they see (full state, cart-only, pole-only). The feature layer is updated by $\boldsymbol{\delta}_z = B\mathbf{e}$ with $\mathbf{e} \in \mathbb{R}^5$, so five parallel learning loops send different errors through B to the same feature representation. Biologically, this corresponds to the full picture of the learning-loops hypothesis: *multiple learning modules* (five critics) that evaluate *different aspects* of the task (e.g. different state views or outcome types), with *different* learning signals ($e_1, \ldots, e_5$) routed to different features via B. The trainable combination w that mixes head outputs for action selection can be seen as a simple stand-in for how the brain might combine multiple value signals to choose actions. The structure is chosen so that we can ask: can a shared feature layer, updated only by a fixed routing of five heterogeneous errors, support a policy that combines five different "critics"? **Gain over Mixed 3F:** (i) Explicitly different *aspects* (different targets and state views per head), not just a reshaped single reward signal; (ii) multiple critics that can in principle specialize (e.g. cart vs. pole, or short vs. long horizon); (iii) a trainable combination for action selection, moving toward "dynamic" use of multiple loops depending on what the policy needs; (iv) the closest match to the biological picture of heterogeneous dopamine and segregated loops (different signals to different targets, with the possibility of combining them for behaviour).

Summary. In light of the original idea of DA learning loops—multiple learning modules, different signals to different targets, fixed or structured routing, and (ideally) dynamic engagement—the progression is: DQN = one loop, one broadcast; Mixed = multiple error dimensions (per action) and fixed routing B to features, with a local feature-layer update; Mixed 3F = blending of error channels at each feature (shared and distinct); Multihead = multiple critics with different aspects and a combined policy. Each step adds a structural or functional ingredient that the learning-loops hypothesis requires, so that we can test both whether the resulting algorithms work and how they relate to the biology.

11.5 Discussion

The multihead algorithm implements the core of Aim 3.2 in a simplified form: *multiple critics* (five value functions with different targets and state views) that evaluate *different aspects* (reward, uprightness, horizon, cart vs. pole), and a *trainable combination* w for action selection. The mixed learning signal $\boldsymbol{\delta}_z = B\mathbf{e}$ provides multiple parallel learning loops to the feature layer. What is not yet implemented is (i) *unsupervised* choice of aspects from environment statistics, and (ii) *dynamic* architecture (number of critics depending on task complexity). Adding those would bring the implementation closer to the full Aim 3.2 vision.